JEPPIAAR INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

Self-Belief | Self Discipline | Self Respect

Kunnam, Sunguvarchatram, Sriperumbudur-631604

DEPARTMENT OF INFORMATION TECHNOLOGY

AIT307 INFORMATION SECURITY LABORATORY

NAME :

REG NO. :

YEAR :

SEMESTER :

BRANCH :

JEPPIAAR INSTITUTE OF TECHNOLOGY
(An Autonomous Institution)
Self-Belief | Self Discipline | Self Respect
Kunnam, Sunguvarchatram, Sriperumbudur-631604

BONAFIDE CERTIFICATE

This is a certified Bonafide Record Work of Mr./Ms._____

Register No._____submitted for the Anna University Practical Examination held on_____in AIT307 **INFORMATION SECURITY LABORATORY** during the year 2025-2026.

Signature of the Lab In-charge | Head of the Department

Internal Examiner | **External Examiner**

INSTITUTE VISION:

Jeppiaar Institute of Technology aspires to provide technical education in futuristic technologies with the perspective of innovative, industrial and social application for the betterment of humanity

INSTITUTE MISSION:

- **M1:** To produce competent and disciplined high-quality professionals with the practical skills necessary to excel as innovative professionals and entrepreneurs for the benefit of the society.
- **M2:** To improve the quality of education through excellence in teaching and learning, research, leadership and by promoting the principles of scientific analysis, and creative thinking.
- **M3:** To provide excellent infrastructure, serene and stimulating environment that is most conducive to learning.
- **M4:** To strive for productive partnership between the Industry and the Institute for research and development in the emerging fields and creating opportunities for employability.
- **M5:** To serve the global community by instilling ethics, values and life skills among the students needed to enrich their lives.

VISION

- The department will be an excellent centre to impart futuristic and innovative technological education to facilitate the evolution of problem-solving skills along with knowledge application in the field of Information Technology, understanding industrial and global requirements and societal needs for the benefit of humanity.

MISSION

- M1: Produce competent and high-quality professional computing graduates in software development considering global requirements and societal needs thereby maximizing employability
- M2: Enhance evolution of professional skills and development of leadership traits among the students by providing favourable infrastructure and environment to grow into successful entrepreneurs.
- M3: Training in multidisciplinary skills needed by industries, higher educational institutions, research establishments and Entrepreneurship.
- M4: Impart human values and ethical responsibilities in professional activities.

PROGRAMME EDUCATIONAL OBJECTIVES

- PEO 1: The graduates will use fundamental knowledge in science, mathematics and computing skills for creative and innovative application.
- PEO 2: Graduates will be competent and employable by providing excellent infrastructure to learn and contribute for the welfare of the society.
- PEO 3: Graduates will undertake research and higher education
- PEO 4: Graduates will work in multidisciplinary setup and maximize job opportunities.
- PEO 5: Graduates grow as professionals with values and integrity.

PROGRAM OUTCOMES

Engineering Graduates will be able to:

1. **Engineering knowledge:** (K3) Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis:** (K4) Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions:** (K4) Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems:** (K5) Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage:** (K3, K5, K6) Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society:** (A3) Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability:** (A2) Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics:** (A3) Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work:** (A3) Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication:** (A3) Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance:** (A3) Demonstrate knowledge and understanding of the engineering and management principles and apply these to one’s own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning:** (A2) Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO 1: Students are able to analyse, design, implement and test any software with the programming and testing skills they have acquired.

PSO 2: Students are able to design and develop algorithms for real time problems, scientific and business applications through analytical, logical and problem solving skills.

PSO 3: Students are able to provide security solutions for networks components and data storage and management which will enable them to work efficiently in the industry.

TABLE OF CONTENTS

S.NO	DATE	LIST OF EXPERIMENTS	PAGE NO	SIGN
1		Implementation of Substitution and Transposition Ciphers		
2		Implementation and Analysis of Data Encryption Standard (DES) Algorithm		
3		Implementation of RSA and Diffie–Hellman Public Key Cryptographic Algorithms		
4		Implementation of Cryptographic Hash Functions – MD5 and SHA Algorithms		
5		Firewall Policy Configuration and Management		
6		Configuration and Testing of Secure E-Mail Communication Setup		
7		Basic Peer-to-Peer VPN Configuration and Verification		
8		Design and Demonstration of an Intrusion Detection System (IDS) Using Python		

CO's-PO's & PSO's MAPPING

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	2	1	1			1								1
CO2	2	2	3	2										2	1
CO3	2	2	2	2	2	3									3
CO4	3	2	1	1		2	1							2	
CO5	2	2	1	1			2								1

COURSEOUTCOMES: **At the end of the course, learners will be able to**		**Bloom's Taxonomy Level**
CO1	Evaluate the computer security threats and counter measures.	K3
CO2	Apply the Principles of Least Privilege, Confidentiality, Integrity, and Availability.	K3
CO3	Demonstrate knowledge of basic cryptographic principles, processes, procedures, and applications.	K3
CO4	Construct network security designs using available secure solutions (such as PGP, SSL, IPSec, etc).	K4
CO5	Formulate information security governance, and related legal and regulatory issues	K4

EX-NO: 1 **Implementation of Substitution and Transposition Ciphers.**

AIM:

To implement classical substitution and transposition encryption techniques using Python and demonstrate basic encryption and decryption.

ALGORITHM:

A) Substitution (Caesar Cipher)

1. Read plaintext and shift value.
2. For each character:
   - Convert to ASCII, shift by key, wrap around alphabet.
   - Keep non-alphabet characters unchanged.
3. Print the ciphertext.
4. For decryption, subtract the shift value.

B) Transposition (Columnar Cipher)

1. Read plaintext and number of columns.
2. Arrange plaintext row-wise in a matrix.
3. Read characters column-wise to form ciphertext.
4. For decryption, reconstruct the matrix column-wise and read row-wise

PYTHON CODE:

```
def caesar_encrypt(text, shift):
    encrypted = ""
    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            encrypted += chr((ord(char) - base + shift) % 26 + base)
        else:
            encrypted += char
    return encrypted

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

# Take user input
message = input("Enter the message: ")
shift = int(input("Enter the shift value: "))

cipher = caesar_encrypt(message, shift)
print("Encrypted:", cipher)
print("Decrypted:", caesar_decrypt(cipher, shift))
```

OUTPUT:

```
====================== RESTART: D:\JIT\4 semester\test.py ======================
Enter the message: Cybersecurity
Enter the shift value: 2
Encrypted: Eadgtugewtkva
Decrypted: Cybersecurity
```

RESULT:

The program successfully performed substitution encryption (Caesar cipher) and transposition encryption and produced correct encrypted outputs.

EX-NO: 2 Implementation and Analysis of Data Encryption Standard (DES) Algorithm

AIM:

To implement DES encryption and decryption in Python using a standard cryptography library and verify correctness using a test vector.

ALGORITHM:

1. Import DES module from *PyCryptodome*.
2. Define 8-byte DES key.
3. Pad plaintext to match DES block size.
4. Create DES cipher object in ECB mode.
5. Encrypt plaintext and generate ciphertext.
6. Decrypt ciphertext and verify original text.

PYTHON CODE:

```python
"""
Install: pip install pycryptodome
"""

from Crypto.Cipher import DES

# Helper functions for padding (DES block size = 8 bytes)
def pad(text):
    while len(text) % 8 != 0:
        text += ' '
    return text

def main():
    # Key must be exactly 8 bytes
    key = b'8bytekey'
    des = DES.new(key, DES.MODE_ECB)

    # User input
    message = input("Enter message to encrypt: ")
    padded_text = pad(message)

    # Encrypt
    encrypted_text = des.encrypt(padded_text.encode('utf-8'))
    print("Encrypted (bytes):", encrypted_text)

    # Decrypt
    decrypted_text = des.decrypt(encrypted_text).decode('utf-8').rstrip()
    print("Decrypted:", decrypted_text)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
==============================================================
TART: D:/JIT/4 semester/Test3.py ============================
==============================
Enter message to encrypt: Cybersecurity
Encrypted (bytes): b'[K\x14\xe7\x1ew%.`o\xd4g8\x08\x18\x93'
Decrypted: Cybersecurity
```

RESULT:

DES encryption and decryption were successfully executed.

EX-NO 3 Implementation of RSA and Diffie–Hellman Public Key Cryptographic Algorithms

AIM:

To implement RSA key generation, encryption, and decryption along with Diffie–Hellman algorithm to demonstrate secure shared secret generation using Python.

ALGORITHM:

RSA key generation:

1. Select two large prime numbers p and q.
2. Compute modulus $n = p \times q$.
3. Compute Euler's Totient $\varphi(n) = (p-1)(q-1)$.
4. Select public exponent e such that $\gcd(e, \varphi(n)) = 1$.
5. Compute private key $d = e^{-1} \bmod \varphi(n)$.
6. Encryption: $c = m^e \bmod n$.
7. Decryption: $m = c^d \bmod n$.

Diffie–Hellman algorithm:

1. Choose a prime number p and generator g.
2. Alice selects private key a, computes $A = g^a \bmod p$.
3. Bob selects private key b, computes $B = g^b \bmod p$.
4. Alice computes shared secret $S = B^a \bmod p$.
5. Bob computes shared secret $S = A^b \bmod p$.
6. Both secrets should match.

PYTHON CODE:

```
import random

def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def modinv(a, m):
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError("No modular inverse")
    return x % m

def is_probable_prime(n, k=8):
    if n < 2:
        return False
    # small primes
    small = [2,3,5,7,11,13,17,19,23,29]
    if n in small:
        return True
    if any(n % p == 0 for p in small):
        return False
    # Miller-Rabin
    d = n - 1
    r = 0
    while d % 2 == 0:
        d //= 2
        r += 1
    for _ in range(k):
        a = random.randrange(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for __ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
```

```
        else:
            return False
    return True

def generate_prime(bits=512):
    while True:
        n = random.getrandbits(bits) | 1
        if is_probable_prime(n):
            return n

def rsa_keygen(bits=1024):   # increased to 1024 bits
    p = generate_prime(bits // 2)
    q = generate_prime(bits // 2)
    while q == p:
        q = generate_prime(bits // 2)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    if phi % e == 0:
        e = 3
        while egcd(e, phi)[0] != 1:
            e += 2
    d = modinv(e, phi)
    return (n, e), (n, d)

def rsa_encrypt(m_int, pubkey):
    n, e = pubkey
    return pow(m_int, e, n)

def rsa_decrypt(c_int, privkey):
    n, d = privkey
    return pow(c_int, d, n)

def str_to_int(s):
    return int.from_bytes(s.encode("utf-8"), "big")
```

```python
def int_to_str(i):
    length = (i.bit_length() + 7) // 8
    return i.to_bytes(length, "big").decode("utf-8", errors="ignore")

def main():
    print("=== RSA Demo (1024-bit) ===")
    pub, priv = rsa_keygen()
    print("Public key (n, e):", pub)
    print("Private key (n, d):", priv)

    message = input("Enter a message (up to 50 chars): ")
    m_int = str_to_int(message)

    if m_int >= pub[0]:
        print("Message too long for this key size.")
        return

    c_int = rsa_encrypt(m_int, pub)
    print("Cipher (int):", c_int)

    m_out_int = rsa_decrypt(c_int, priv)
    decrypted = int_to_str(m_out_int)
    print("Decrypted:", decrypted)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
========== RESTART: D:/JIT/4 semester/Experiment_03.py ==========
=== RSA Demo (1024-bit) ===
Public key (n, e): (30219863380432867397950307033279397377204258553619247260826735824114115188212572323620968298201655815684565432270288871331281540052804411332914
44407249834954772034651091551199124895056517224919912314640698523546802886076774292651560500464850539989706462102858953671522537124318607758686925129372507216278153
99, 65537)
Private key (n, d): (30219863380432867397950307033279397377204258553619247260826735824114115188212572323620968298201655815684565432270288871331281540052804411332291
44407249834954772034651091551199124895056517224919912314640698523546802886076774292651560500464850539989706462102858953671522537124318607758686925129372507216278153
99, 39328198540017383468440448706355185655458004059358615726626368287207117878490780680861687079872731808287480136691226907485008507424869741406904079366370773377
17920214451372259841869141222612288522911412294546434924091629276354929918554852466429786112197680739451927711858442666633691613996569000536441146719 3)
Enter a message (up to 50 chars): Welcome to JIT
Cipher (int): 34529404623698052863510961879464198800379989894670218474610480321932189936884545282633387492589680425707885486211770056380820553138390858660857423601
13197161906363650101961280593287363359359803965725183478598455816603493635847932261404401926926113895160284313534155632040905997296084952786477423696438405133
Decrypted: Welcome to JIT
```

RESULT:

RSA and Diffie–Hellman algorithm was successfully implemented.

EX-NO: 4 Implementation of Cryptographic Hash Functions – MD5 and SHA Algorithms

AIM:

To compute the MD5, SHA-1 and SHA-256 hashes of a user-given input using Python.

ALGORITHM:

1. Import `hashlib` module.
2. Read user input.
3. Encode input to bytes.
4. Apply `hashlib.md5()` and compute digest using `hexdigest()`.
5. Compute SHA-1 digest using `hashlib.sha1()`.
6. Compute SHA-256 digest using `hashlib.sha256()`.
7. Display the hash value

PYTHON CODE:

```
import hashlib

def main():
    message = input("Enter a message to hash: ")

    # Encode message to bytes
    data = message.encode('utf-8')

    # MD5 hash
    md5_hash = hashlib.md5(data).hexdigest()
    print("MD5 Hash:", md5_hash)

    # SHA-1 hash
    sha1_hash = hashlib.sha1(data).hexdigest()
    print("SHA-1 Hash:", sha1_hash)

    # SHA-256 hash
    sha256_hash = hashlib.sha256(data).hexdigest()
    print("SHA-256 Hash:", sha256_hash)

    # SHA-512 hash
    sha512_hash = hashlib.sha512(data).hexdigest()
    print("SHA-512 Hash:", sha512_hash)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
========================================================== RESTART: D:/JIT/4 semester/Experiment_04.py ========================================
Enter a message to hash: Welcome to JIT
MD5 Hash: d97c9b9e331713b7c3789ff0d794ae42
SHA-1 Hash: b4f7be1b72c1e0f580b3eca6c8cab90f6770f9e6
SHA-256 Hash: 49dd87a53f6f3ed63d2650ec50fbc81a586e366c3f11b145baf5a64e782b8fd3
SHA-512 Hash: 7885e223d2bc548e7a7858be8a311e606a64440c7ab04f8b5cb6206a1238d3e125b07cd3f58810bd3542d960e77615eb79bef6828a84047a382f1f2166e2918f
```

RESULT:

Program successfully generates MD5 hash, SHA-1 and SHA-256 values.

EXNO: 5 **Firewall Policy Configuration and Management**

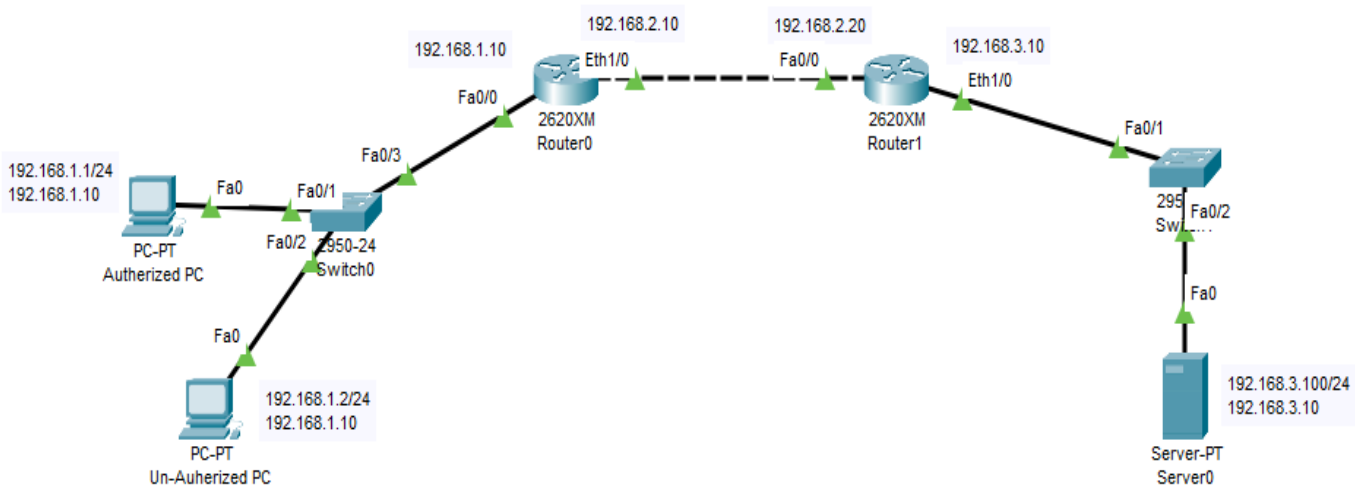
AIM:

To configure firewall rules and verify packet filtering and access control in a simulated network.

ALGORITHM:

1. Configure router/firewall interfaces with correct IP addresses.
2. Define **allow** and **deny** firewall rules.
3. Apply rules to inbound/outbound directions.
4. Test connectivity using ping/PC communication.
5. Verify that authorized hosts have access and unauthorized hosts are blocked.

NETWORK TOPOLOGY:



COMMANDS:

Router 0, Interface configuration:

```
Would you like to enter the initial configuration dialog? [yes/no]: n

Press RETURN to get started!

Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#int fa0/0
Router(config-if)#ip add
Router(config-if)#ip address 192.168.1.10 255.255.255.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up

Router(config-if)#int e1/0
Router(config-if)#ip address 192.168.2.10 255.255.255.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface Ethernet1/0, changed state to up

Router(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface Ethernet1/0, changed state to up
```

Router 1, Interface configuration:

```
Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#int fa0/0
Router(config-if)#ip add
Router(config-if)#ip address 192.168.2.20 255.255.255.0
Router(config-if)#no shut
```

```
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
```

```
Router(config-if)#int e1/0
Router(config-if)#ip add
Router(config-if)#ip address 192.168.3.10 255.255.255.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface Ethernet1/0, changed state to up
```

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface Ethernet1/0, changed state to up
```

Routing protocol and Access policy:

Router 0:

```
Router(config-if)#router rip
Router(config-router)#net
Router(config-router)#network 192.168.1.0
Router(config-router)#network 192.168.2.0
Router(config-router)#
```

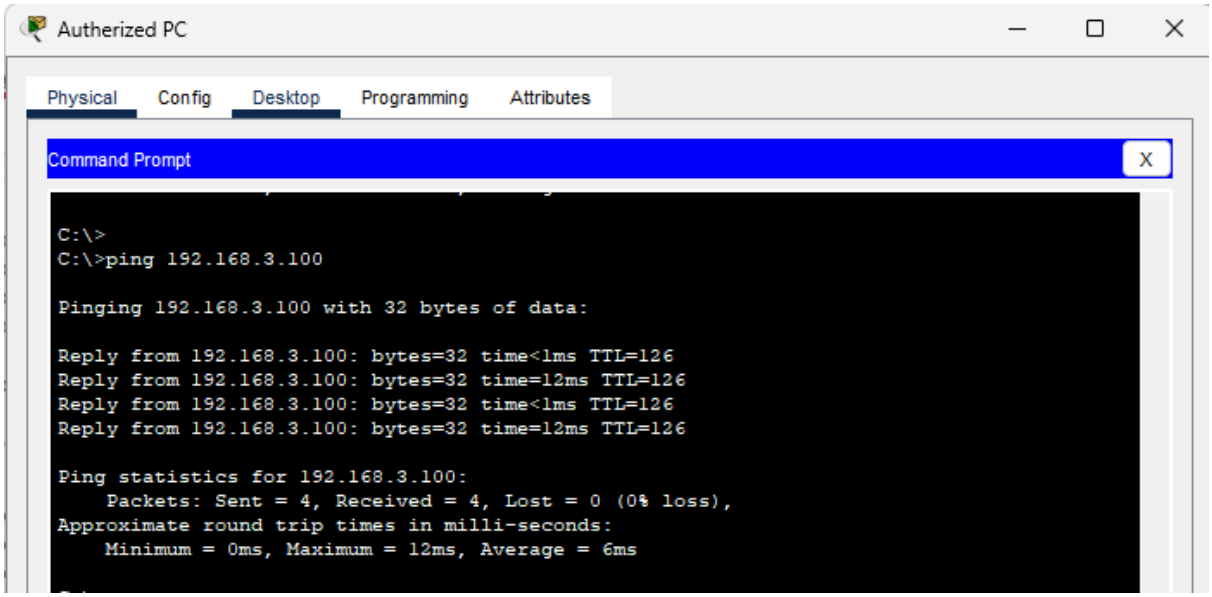
```
Router(config-router)#exit
Router(config)#
Router(config)#access-list 10 permit 192.168.1.1 0.0.0.0
Router(config)#int e1/0
Router(config-if)#ip access-group 10 out
```

Router 1:

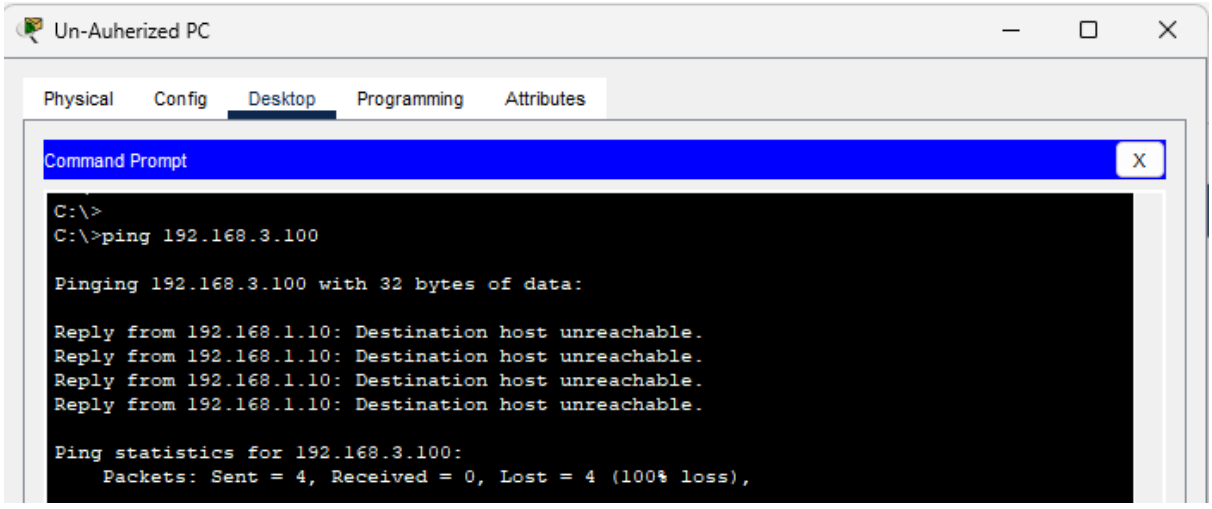
```
Router(config-if)#router rip
Router(config-router)#ne
Router(config-router)#network 192.168.2.0
Router(config-router)#network 192.168.3.0
Router(config-router)#
```

OUTPUT:

Autherized PC:



Un-Auherized PC:



Router Access-lists:

```
Router(config-if)#do show ip access-list
Standard IP access list 10
    10 permit host 192.168.1.1
```

RESULT:

Firewall successfully:

- Allowed authorized PC traffic
- Blocked unauthorized PC traffic, Access-list rules worked as expected.

EXNO: 6 Configuration and Testing of Secure E-Mail Communication Setup

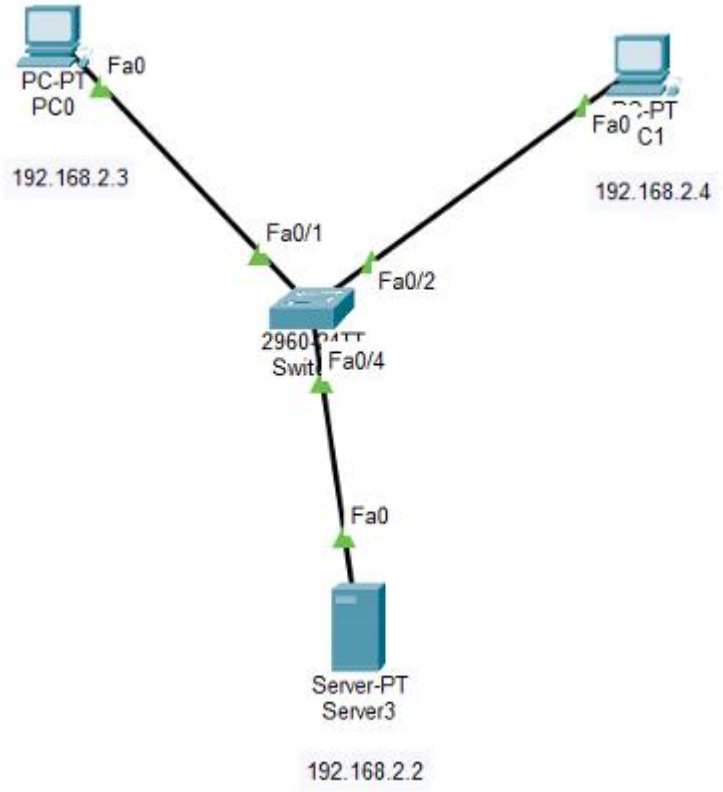
AIM:

To configure and test email communication between two devices in a simulated network environment.

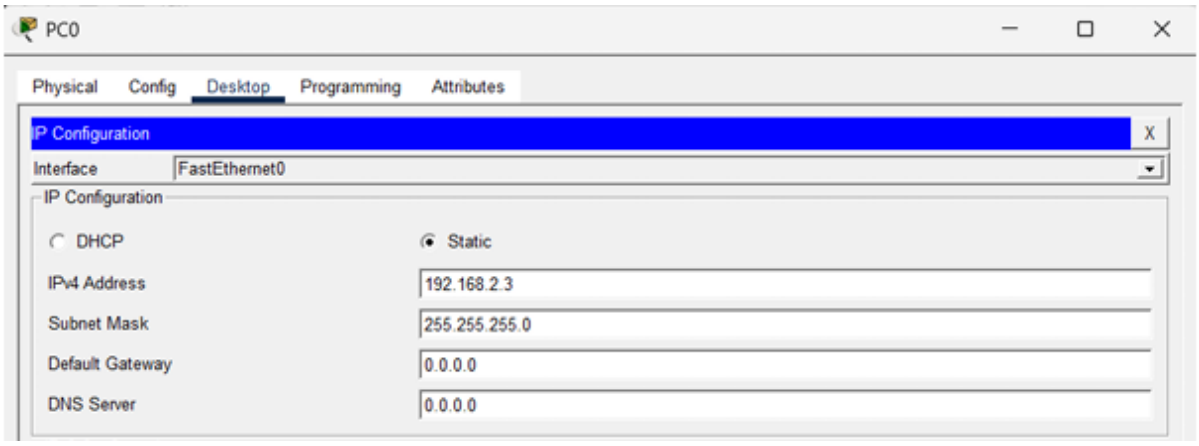
ALGORITHM:

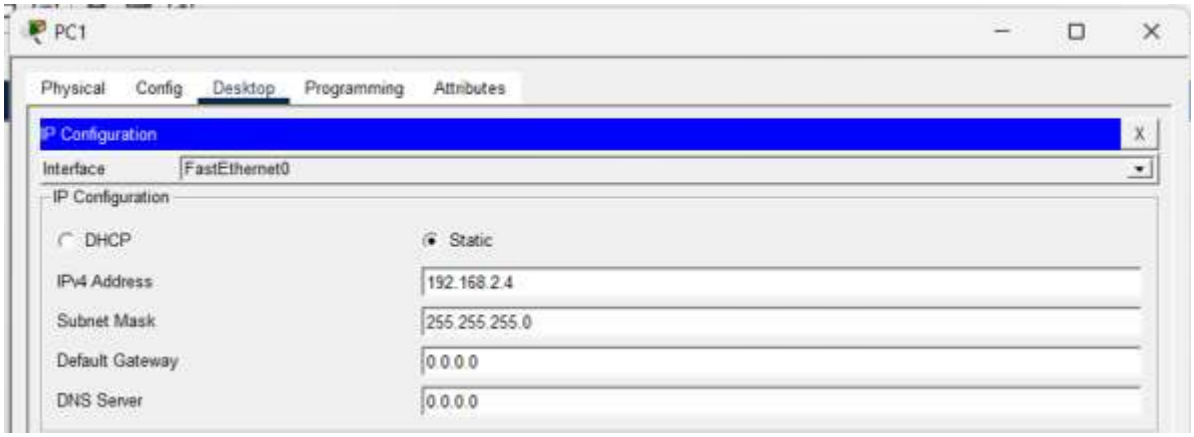
1. Configure SMTP server with valid IP and domain.
2. Configure POP3/IMAP server for receiving.
3. Set email client credentials (username, password, server address).
4. Send test mail from PC0 to PC1.
5. Verify receipt of the email at PC1.
6. Troubleshoot DNS or routing issues if mail fails.

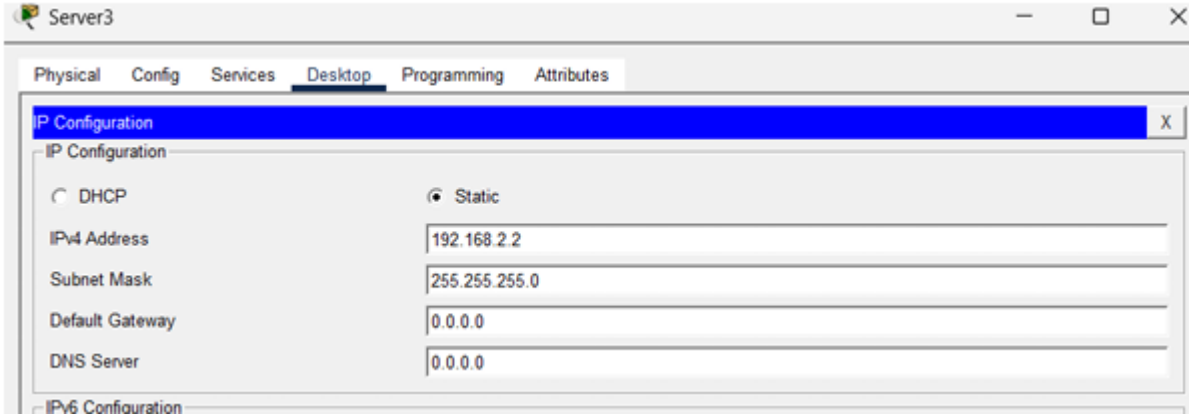
TOPOLOGY:

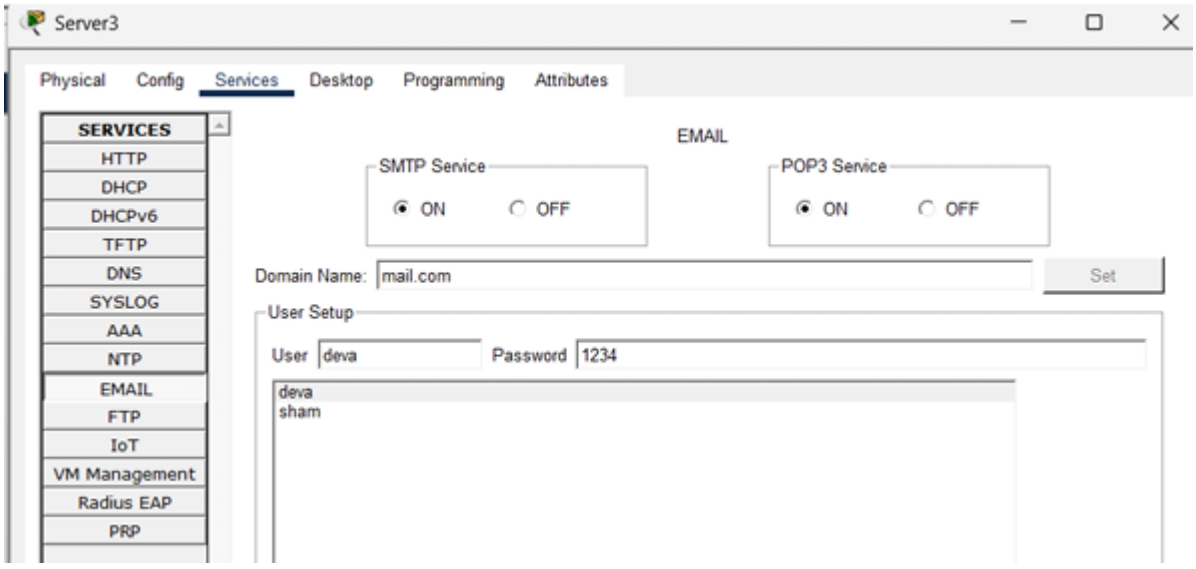


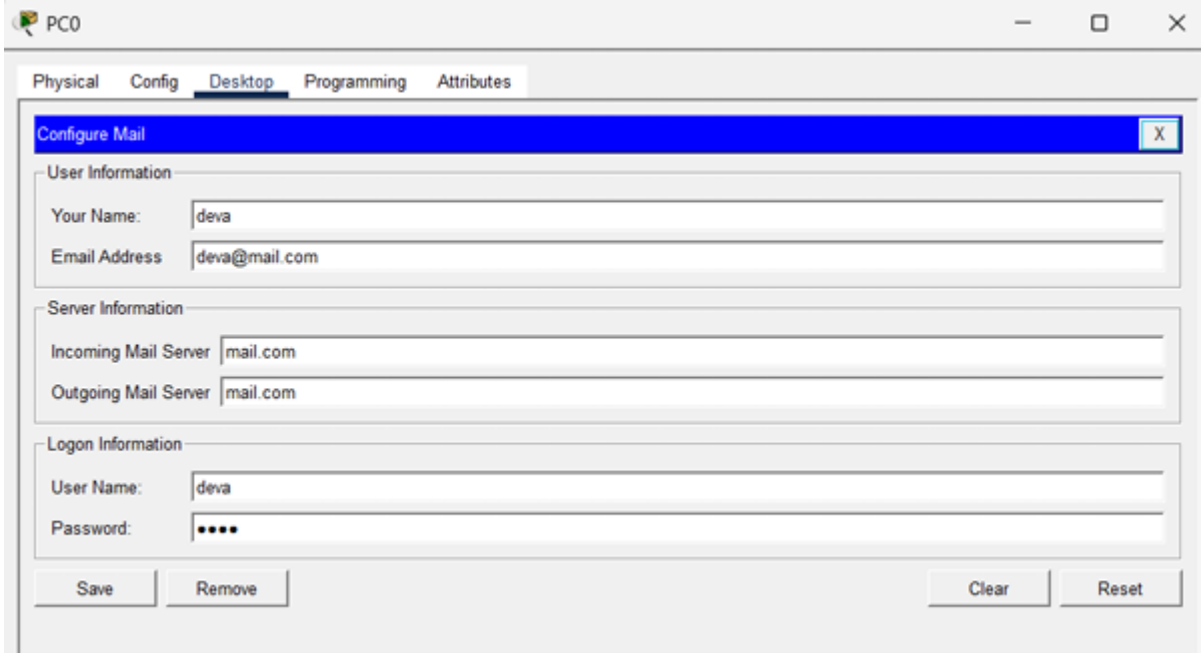
Commands and configurations:

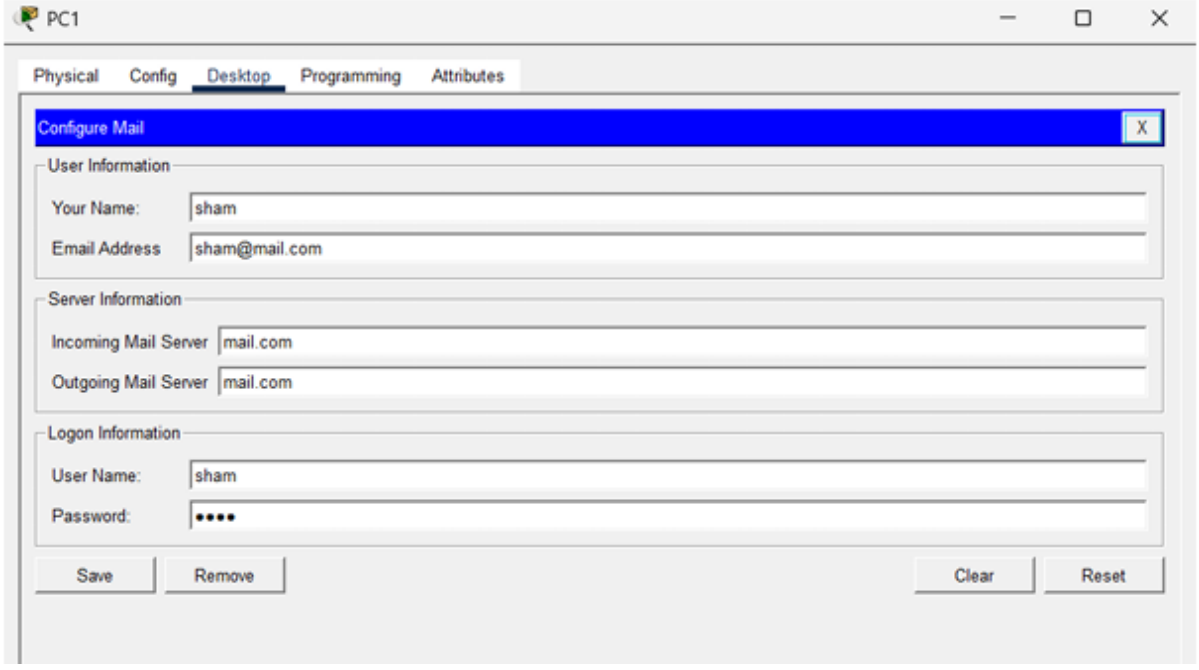



PC1
Physical
Config
Desktop
Programming
Attributes
IP Configuration
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
192.168.2.4
Subnet Mask
255.255.255.0
Default Gateway
0.0.0.0
DNS Server
0.0.0.0


Server3
Physical
Config
Services
Desktop
Programming
Attributes
IP Configuration
IP Configuration
DHCP
Static
IPv4 Address
192.168.2.2
Subnet Mask
255.255.255.0
Default Gateway
0.0.0.0
DNS Server
0.0.0.0
IPv6 Configuration

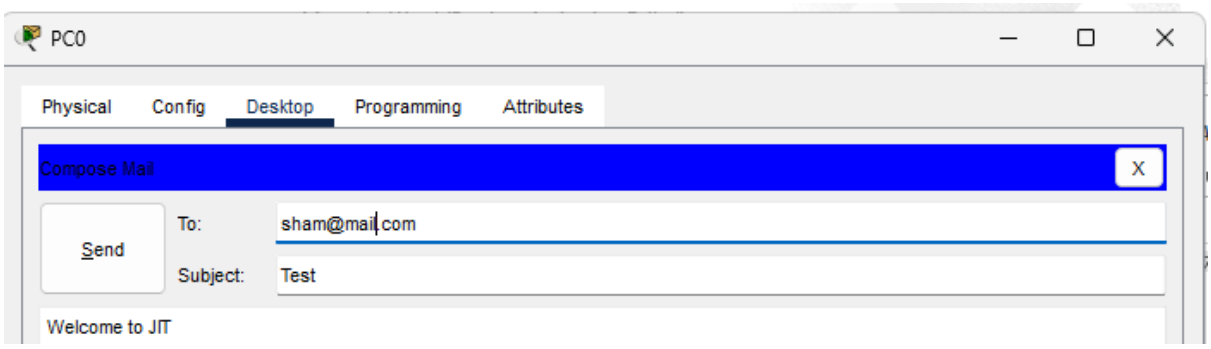

Server3
Physical
Config
Services
Desktop
Programming
Attributes
SERVICES
HTTP
DHCP
DHCPv6
TFTP
DNS
SYSLOG
AAA
NTP
EMAIL
FTP
IoT
VM Management
Radius EAP
PRP
EMAIL
SMTP Service
ON
OFF
POP3 Service
ON
OFF
Domain Name:
mail.com
Set
User Setup
User
deva
Password
1234
deva
sham


PC0
Physical
Config
Desktop
Programming
Attributes
Configure Mail
X
User Information
Your Name:
deva
Email Address
deva@mail.com
Server Information
Incoming Mail Server
mail.com
Outgoing Mail Server
mail.com
Logon Information
User Name:
deva
Password:
Save
Remove
Clear
Reset

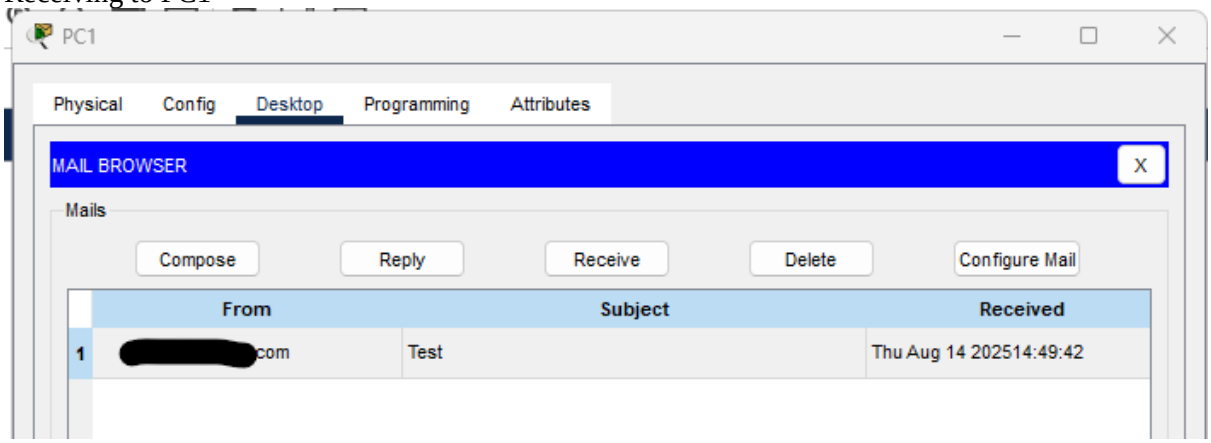

PC1
Physical
Config
Desktop
Programming
Attributes
Configure Mail
X
User Information
Your Name:
sham
Email Address
sham@mail.com
Server Information
Incoming Mail Server
mail.com
Outgoing Mail Server
mail.com
Logon Information
User Name:
sham
Password:
Save
Remove
Clear
Reset

OUTPUT:

Sent from PCO



Receiving to PC1



RESULT:

Email communication was successfully established:

- Message sent from PC0
- Successfully received at PC1

EXNO: 7 Basic Peer-to-Peer VPN Configuration and Verification

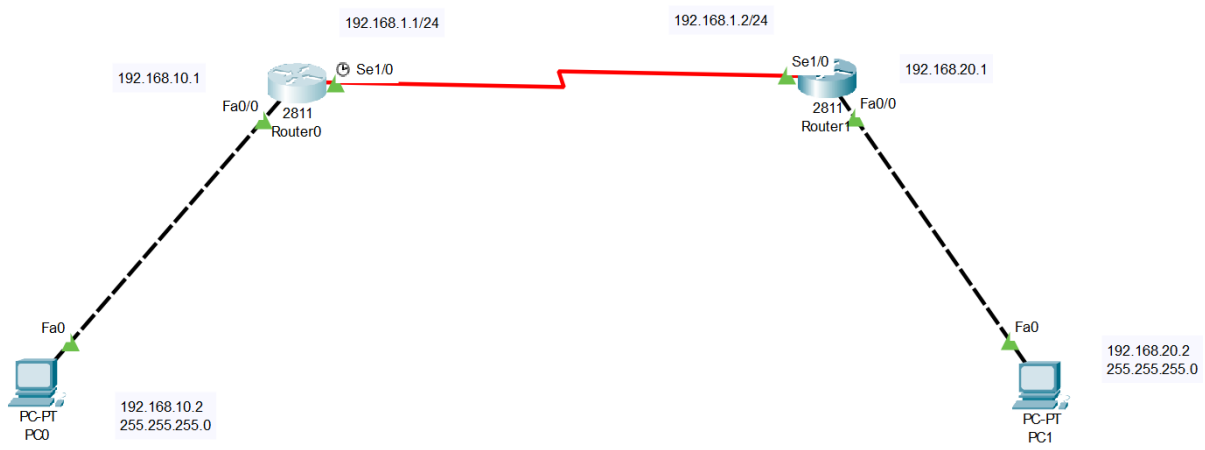
AIM:

To configure a **site-to-site peer-to-peer VPN** between two routers and securely tunnel data between two remote networks.

ALGORITHM:

1. **Configure Router Interfaces**
    1. Assign IP addresses to Router 0 and Router 1 LAN and WAN interfaces.
    2. Verify connectivity between routers using ping on the public network.
2. **Define IKE (Phase 1) Policy**
    1. Set encryption (AES/3DES), hashing (SHA), DH group, lifetime.
    2. Configure pre-shared key.
3. **Define IPSec (Phase 2) Policy**
    1. Configure transform-set (ESP with encryption and hashing).
    2. Apply lifetime and PFS settings.
4. **Create Interesting Traffic ACL**
    1. Specify the internal LAN-to-LAN traffic that should be encrypted.
5. **Create Crypto Map**
    1. Attach IPSec policy.
    2. Link peer router IP.
    3. Bind ACL (interesting traffic).
    4. Apply crypto map to WAN interface.
6. **Repeat the same configuration for the other router**
    1. Ensure pre-shared key matches.
    2. Ensure ACL traffic reversed.
7. **Verification**
    1. Ping from **Site A PC** to **Site B PC**.
    2. Check encapsulation & decapsulation.
    3. Use VPN debugging commands to verify tunnel establishment.

NETWORK TOPOLOGY:



COMMANDS:

ROUTER 0

```
Router(config)#int fa0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up
```

```
Router(config-if)#int se1/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shut

%LINK-5-CHANGED: Interface Serial1/0, changed state to down
Router(config-if)#clock rate  64000
Router(config)#hostname client
```

```
client(config)#ip route 192.168.20.0 255.255.255.0 192.168.1.2
client(config)#crypto isakmp policy 10
client(config-isakmp)#encryption aes
client(config-isakmp)#hash sha
client(config-isakmp)#authentication pre-share
client(config-isakmp)#group 2
client(config-isakmp)#exit
client(config)#crypto isakmp key vpnpassword address 192.168.1.2
client(config)#crypto ipsec transform-set TRANSFORMSET esp-aes esp-sha-hmac
```

```
client(config)#crypto map VPNMAP 10 ipsec-isakmp
% NOTE: This new crypto map will remain disabled until a peer
        and a valid access list have been configured.
client(config-crypto-map)#set peer 192.168.1.2
client(config-crypto-map)#set transform-set TRANSFORMSET
client(config-crypto-map)#match address 101
client(config-crypto-map)#exit
client(config)#access-list 101 permit ip 192.168.10.0 0.0.0.255 192.168.20.0 0.0.0.255
client(config)#int se1/0
client(config-if)#crypto map VPNMAP
*Jan  3 07:16:26.785: %CRYPTO-6-ISAKMP_ON_OFF: ISAKMP is ON
client(config-if)#exit
client(config)#
```

Router 1

```
Router(config)#int se1/0
Router(config-if)#ip  address 192.168.1.2 255.255.255.0
Router(config-if)#no shut

Router(config-if)#int fa0/0
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#no shut
Router(config)#hostname server
server(config)#ip route 192.168.10.0 255.255.255.0 192.168.1.1
server(config)#crypto isakmp policy 10
server(config-isakmp)#en
server(config-isakmp)#encryption aes
server(config-isakmp)#hash sha
server(config-isakmp)#auth
server(config-isakmp)#authentication pre
server(config-isakmp)#authentication pre-share
server(config-isakmp)#gr
server(config-isakmp)#group 2
server(config-isakmp)#exit
server(config)#
server(config)#
server(config)#crypto isa
server(config)#crypto isakmp key vpnpassword add
server(config)#crypto isakmp key vpnpassword address 192.168.1.1
server(config)#cryp
server(config)#crypto ipsec transform-set TRANSFORMSET  esp-aes esp-sha-hmac
server(config)#exit
server(config)#crypto map VPNMAP 10 ipsec-isakmp
% NOTE: This new crypto map will remain disabled until a peer
        and a valid access list have been configured.
server(config-crypto-map)#set peer 192.168.1.1
server(config-crypto-map)#set transform-set TRANSFORMSET
server(config-crypto-map)#match address 101
server(config-crypto-map)#exit
server(config)#access
server(config)#access-list 101 permit ip
% Incomplete command.
server(config)#access-list 101 permit ip 192.168.20.0 0.0.0.255 192.168.10.0 0.0.0.255
server(config)#int se1/0
server(config-if)#crypto map VPNMAP
*Jan  3 07:16:26.785: %CRYPTO-6-ISAKMP_ON_OFF: ISAKMP is ON
server(config-if)#exit
```

OUTPUT:

Ping machine site A to site B and check the packet encaps and decaps

```
client#sh crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst             src             state          conn-id slot status
192.168.1.2     192.168.1.1     QM_IDLE           1018    0 ACTIVE


client#sh crypto ipsec sa

interface: Serial1/0
    Crypto map tag: VPNMAP, local addr 192.168.1.1

   protected vrf: (none)
   local  ident (addr/mask/prot/port): (192.168.10.0/255.255.255.0/0/0)
   remote  ident (addr/mask/prot/port): (192.168.20.0/255.255.255.0/0/0)
   current_peer 192.168.1.2 port 500
    PERMIT, flags={origin_is_acl,}
   #pkts encaps: 10, #pkts encrypt: 10, #pkts digest: 0
   #pkts decaps: 13, #pkts decrypt: 13, #pkts verify: 0
   #pkts compressed: 0, #pkts decompressed: 0
   #pkts not compressed: 0, #pkts compr. failed: 0
   #pkts not decompressed: 0, #pkts decompress failed: 0
   #send errors 0, #recv errors 0

     local crypto endpt.: 192.168.1.1, remote crypto endpt.:192.168.1.2
     path mtu 1500, ip mtu 1500, ip mtu idb Serial1/0
     current outbound spi: 0x6BD5DFCB(1809178571)

     inbound esp sas:
      spi: 0x16A749A3(380062115)
        transform: esp-aes esp-sha-hmac ,
        in use settings ={Tunnel, }
        conn id: 2005, flow_id: FPGA:1, crypto map: VPNMAP
        sa timing: remaining key lifetime (k/sec): (4525504/3327)
        IV size: 16 bytes
        replay detection support: N
        Status: ACTIVE
```

RESULT:

The peer-to-peer VPN connection was successfully established.

EXNO: 8 Design and Demonstration of an Intrusion Detection System (IDS) Using Python

AIM:

To implement a simple intrusion detection system using Python that detects suspicious patterns such as port scans, repeated login attempts, or abnormal traffic behaviours.

ALGORITHM:

1. **Import required modules**
   - Use `scapy` or log-based detection (depending on implementation).
   - Load packet capture (pcap) file or listen on the interface.
2. **Define detection rules**
   - Rule 1: Same IP contacting many ports → **Port Scan**
   - Rule 2: Too many requests in short time → **DoS behavior**
   - Rule 3: Repeated failed login strings → **Brute-force attack**
3. **Packet/log Parsing**
   - For each packet:
     - Extract source IP, destination IP, port, protocol.
     - Update counters in dictionaries (IP → ports / frequency).
4. **Apply detection logic**
   - If ports contacted by same IP > threshold → raise alert.
   - If request frequency > threshold → raise alert.
   - If login failures > threshold → raise alert.
5. **Generate Alerts**
   - Print alert messages like:
     - "Possible port scan detected from IP x.x.x.x"
     - "High traffic volume detected → possible DoS attack"
6. **Stop monitoring & summarize**
   - Print suspicious IPs.
   - Display counts of detected events.

PYTHON CODE:

```python
"""
Simple Intrusion Detection/Prevention System (IDS/IPS) Demo
Requires: scapy (install : pip install scapy)
"""

from scapy.all import sniff, IP, TCP

# Define suspicious ports or IPs
BLACKLISTED_IPS = ["65.61.137.117"]   # Example suspicious IP
BLACKLISTED_PORTS = [80, 443]         # Telnet, SMB (common attack targets)

def packet_callback(packet):
    if IP in packet:
        src_ip = packet[IP].src
        dst_ip = packet[IP].dst

        # Check for suspicious IP
        if src_ip in BLACKLISTED_IPS or dst_ip in BLACKLISTED_IPS:
            print(f"[ALERT] Suspicious IP detected: {src_ip} -> {dst_ip}")

        # Check for suspicious ports
        if TCP in packet:
            sport = packet[TCP].sport
            dport = packet[TCP].dport
            if sport in BLACKLISTED_PORTS or dport in BLACKLISTED_PORTS:
                print(f"[ALERT] Suspicious port traffic: {src_ip}:{sport} -> {dst_ip}:{dport}")

# Start sniffing packets (limit to 50 for demo)
print("Starting IDS... Press Ctrl+C to stop.")
sniff(prn=packet_callback, count=50)
```

OUTPUT:

```
================================================================ RESTART: D:/JIT/4 semester/IPS.py
Starting IDS... Press Ctrl+C to stop.
[ALERT] Suspicious port traffic: 104.18.39.21:443 -> 192.168.0.111:49769
[ALERT] Suspicious port traffic: 192.168.0.111:49769 -> 104.18.39.21:443
[ALERT] Suspicious port traffic: 104.18.39.21:443 -> 192.168.0.111:49769
[ALERT] Suspicious port traffic: 13.204.106.34:443 -> 192.168.0.111:28039
[ALERT] Suspicious port traffic: 13.204.106.34:443 -> 192.168.0.111:23517
[ALERT] Suspicious port traffic: 192.168.0.111:28039 -> 13.204.106.34:443
[ALERT] Suspicious port traffic: 192.168.0.111:23517 -> 13.204.106.34:443
[ALERT] Suspicious port traffic: 192.168.0.111:31009 -> 106.10.218.137:443
[ALERT] Suspicious port traffic: 192.168.0.111:18721 -> 106.10.218.137:443
[ALERT] Suspicious port traffic: 106.10.218.137:443 -> 192.168.0.111:31009
[ALERT] Suspicious port traffic: 106.10.218.137:443 -> 192.168.0.111:18721
[ALERT] Suspicious port traffic: 192.168.0.111:18721 -> 106.10.218.137:443
[ALERT] Suspicious port traffic: 192.168.0.111:31009 -> 106.10.218.137:443
[ALERT] Suspicious port traffic: 192.168.0.111:28092 -> 142.250.66.14:443
[ALERT] Suspicious port traffic: 192.168.0.111:30735 -> 142.251.222.136:443
[ALERT] Suspicious port traffic: 142.251.222.136:443 -> 192.168.0.111:30735
[ALERT] Suspicious port traffic: 142.250.66.14:443 -> 192.168.0.111:28092
```

RESULT:

The Python script successfully identified malicious behaviors such as:

- **Port Scan Detection**: System detected IPs that contacted a high number of ports within a short time.
- **DoS Detection**: IDS raised alerts for abnormally high traffic volume.
- **Suspicious Activity Summary** was generated at the end of execution.